# EvilCorp

*Computer Security*

## Lab Report

**Prepared by:**

Hadid Ramy
Moussous Amir Mehdi
Said Sarah
Ferrah Anis

Group 3 | Computer Security Module | 3rd Year

© 2025 EvilCorp Project — For educational use only

# 1. Introduction

As part of the Computer Security module (3rd year undergraduate), we developed EvilCorp, a C# desktop application (Windows Forms) designed to experimentally illustrate several fundamental concepts in computer security. The tool includes:

- Classical encryption algorithms: Caesar cipher, Hill cipher, Affine cipher;
- Password attack simulations: dictionary attack, brute-force attack;
- A Man-in-the-Middle (MitM) attack simulation;
- A steganography module using the LSB method.

The educational goal is to allow students to concretely understand how these mechanisms work, their limitations, and the vulnerabilities they expose, through an interactive and reproducible hands-on approach.

# 2. EvilCorp — General Overview

EvilCorp is structured into three main modules accessible from a clean Windows Forms interface:

- Encryption / Decryption: testing classical cryptographic algorithms;
- Passwords & Attacks: password-based attack simulations;
- Steganography: hiding and recovering concealed messages.

The user types a message, selects the desired algorithm or attack, and launches the operation via a button. The result is displayed instantly in the interface, making it easy to visualise and understand each method.

# 3. Encryption and Decryption

## 3.1 Caesar Cipher

The Caesar cipher is one of the oldest substitution algorithms. Each letter in the message is shifted by a fixed number of positions in the alphabet, defined by key k.

| | |
|---|---|
| **Formula** | *Encryption: $C = (P + k) \bmod 26$ \| Decryption: $P = (C - k + 26) \bmod 26$* |

C# implementation: the text is traversed character by character. Each letter is converted to an index (0–25), the shift is applied modulo 26, and the resulting character is reconstructed. Non-alphabetic characters are preserved as-is.

**Worked Example (k = 3)**

| Plaintext | P (index) | C = (P+3) mod 26 | Ciphertext letter |
|---|---|---|---|
| H | 7 | 10 | K |
| E | 4 | 7 | H |

| | | | |
|---|---|---|---|
| L | 11 | 14 | O |
| L | 11 | 14 | O |
| O | 14 | 17 | R |

Result: HELLO → KHOOR (with k = 3)

### Limitations

- Very small key space: only 25 possible shifts, trivially broken by brute force.
- Letter frequency distribution is preserved → vulnerable to frequency analysis.
- Strictly pedagogical; never appropriate for real-world security.

## 3.2 Hill Cipher

The Hill cipher is a block encryption algorithm based on linear algebra. The message is split into vectors of size n, then multiplied by an invertible key matrix K modulo 26.

| **Formula** | *Encryption: $C = K \times P \pmod{26}$* | | *Decryption: $P = K^{-1} \times C \pmod{26}$* |
|---|---|---|---|

C# implementation: the key matrix K is stored as an int[,] array. Each block is converted to an integer vector, the matrix product is computed and reduced modulo 26. A separate function computes $K^{-1}$ (modular inverse) for decryption.

### Worked Example — 2×2 Matrix

| Parameter | Value |
|---|---|
| Key matrix K | [[3, 3], [2, 5]] |
| Plaintext | "HI" → [7, 8] |
| Encryption K×P mod 26 | [3×7+3×8, 2×7+5×8] = [45, 54] → [19, 2] |
| Ciphertext | "TC" |
| $K^{-1}$ (mod 26) | [[15, 17], [20, 9]] |
| Verification $K^{-1}$×C mod 26 | [7, 8] → "HI" ✓ |

### Limitations

- Vulnerable to known-plaintext attacks: a few (plaintext, ciphertext) pairs allow K to be recovered by solving a linear system.
- The purely linear structure does not provide the diffusion required by modern standards.

## 3.3 Affine Cipher

The affine cipher generalises the Caesar cipher. The key consists of two parameters a and b, where a must be coprime with 26 to guarantee the existence of a multiplicative inverse.

| Formula | *Encryption: $C = (a \times P + b) \bmod 26$ \| Decryption: $P = a^{-1} \times (C - b) \bmod 26$* |
|---|---|

**Worked Example (a = 5, b = 8)**

| Letter | P | C = (5P+8) mod 26 | Result |
|---|---|---|---|
| A | 0 | (0+8) mod 26 = 8 | I |
| F | 5 | (25+8) mod 26 = 7 | H |
| I | 8 | (40+8) mod 26 = 22 | W |
| N | 13 | (65+8) mod 26 = 21 | V |
| E | 4 | (20+8) mod 26 = 2 | C |

**Limitations**

- Key space limited to 312 valid (a, b) pairs for the Latin alphabet.
- Preserves the statistical structure of the language → vulnerable to frequency analysis.

**Comparative Summary**

| Algorithm | Type | Key size | Brute-force resistance | Freq. analysis resistance |
|---|---|---|---|---|
| Caesar | Mono. substitution | 25 keys | ⚠ Very weak | ⚠ Weak |
| Affine | Mono. substitution | 312 keys | ⚠ Weak | ⚠ Weak |
| Hill (2×2) | Poly. substitution | Matrix mod 26 | ✓ Better | ✓ Better |

# 4. Password Attacks

## 4.1 Password Scenarios Tested

| Scenario | Characteristics | Possible combinations | Estimated brute-force time |
|---|---|---|---|
| 1 | 3 characters from {2,3,4} | $3^3 = 27$ | < 1 ms |
| 2 | 5 digits (0–9) | $10^5 = 100{,}000$ | ~22 ms |
| 3 | 6 chars (letters + digits + symbols) | $\approx 80^6 \approx 2.6 \times 10^{11}$ | Prohibitive |

## 4.2 Dictionary Attack

A dictionary attack tests a list of commonly used passwords rather than enumerating all combinations. This method exploits the fact that most users choose predictable passwords.

### Results Obtained

| Target password | In dictionary? | Result | Attempts before success |
|---|---|---|---|
| 234 | Yes | ✓ Found | 3 |
| password | Yes | ✓ Found | 7 |
| x7#Kp! | No | ✗ Failed | N/A (list exhausted) |

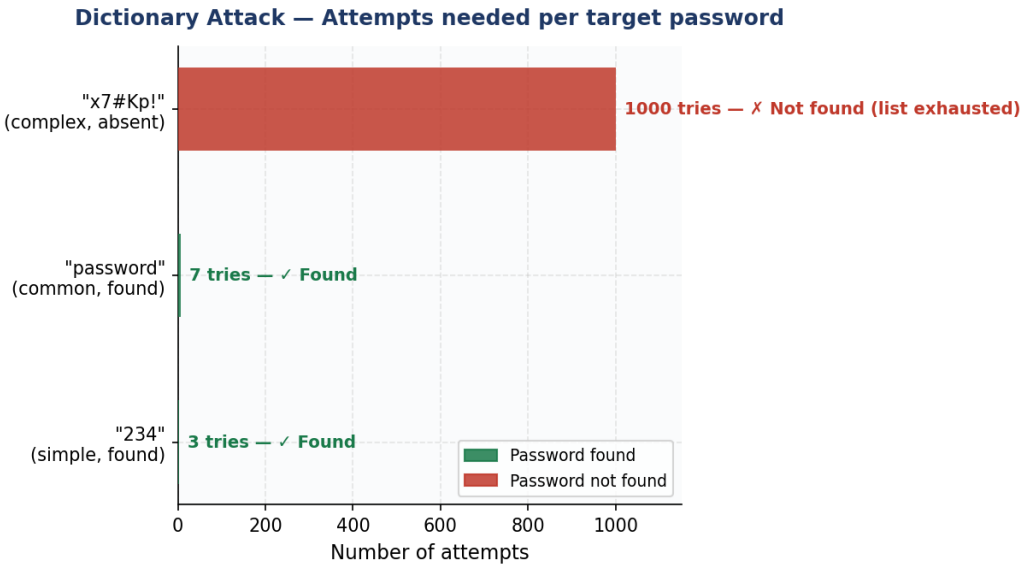


*Figure 1 — Dictionary attack: number of attempts required per target password*

## 4.3 Complexity Analysis — Exponential Growth

The security of a password is directly linked to the size of the search space, which grows exponentially with both its length and the alphabet size. For an alphabet of size b and a password of length n, the number of combinations equals $b^n$.

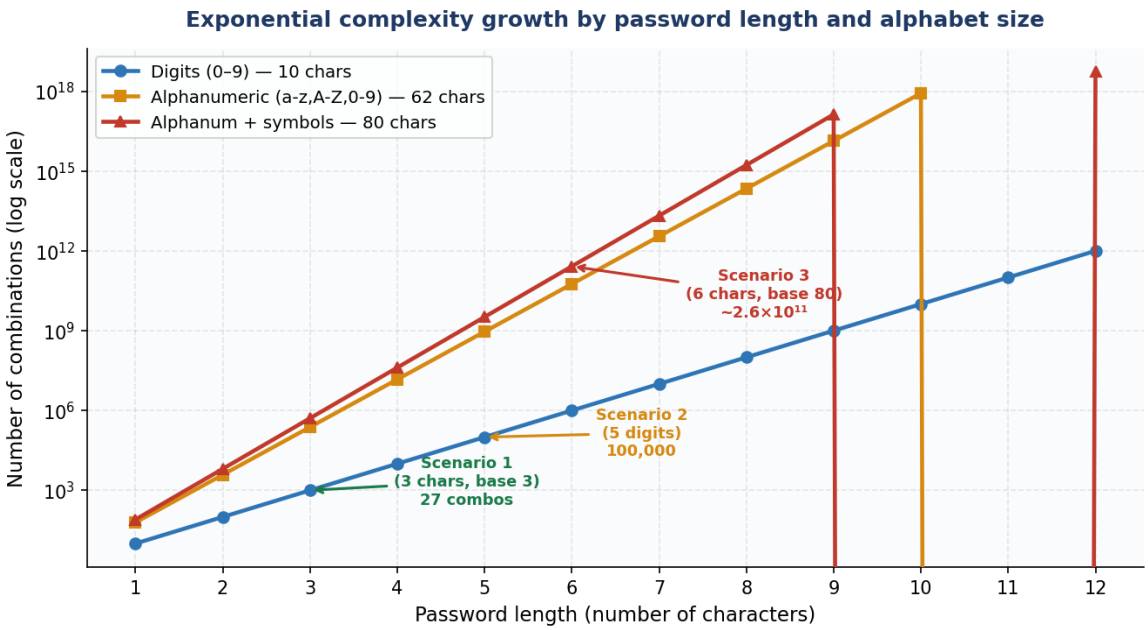


*Figure 2 — Exponential growth of the number of combinations by password length and alphabet size. The three tested scenarios are annotated.*

This graph clearly shows that even a small increase in password length or a wider alphabet causes a combinatorial explosion that makes brute-force attacks completely impractical without significant resources.

## 4.4 Brute-Force Attack — Results and Performance

A brute-force attack systematically generates and tests all possible combinations from a given alphabet. It theoretically guarantees finding the password, but the time required grows exponentially with length and complexity.

### Observed Performance Measurements

| Scenario | Password | Combinations tested | Measured duration |
|---|---|---|---|
| 1 — {2,3,4}, 3 chars | 234 | 27 | < 1 ms |
| 2 — digits, 5 chars | 47291 | 47,292 | ~22 ms |
| 3 — alphanum+sym, 6 chars | a1#Z9! | Not completed | Timeout |

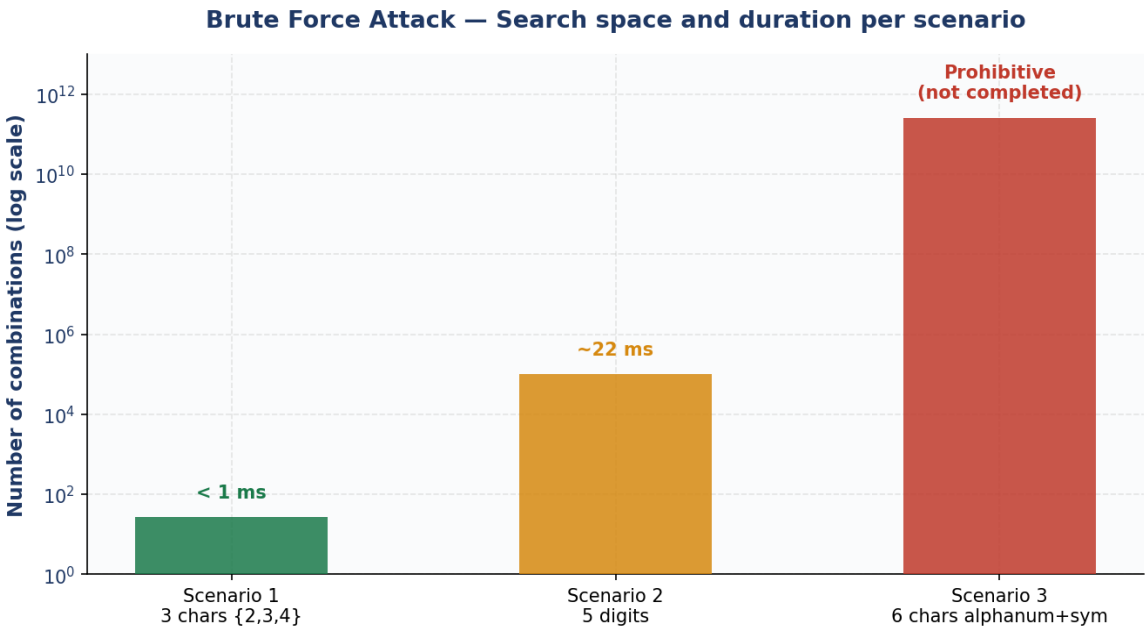


*Figure 3 — Search space (log scale) and measured duration for each brute-force attack scenario*

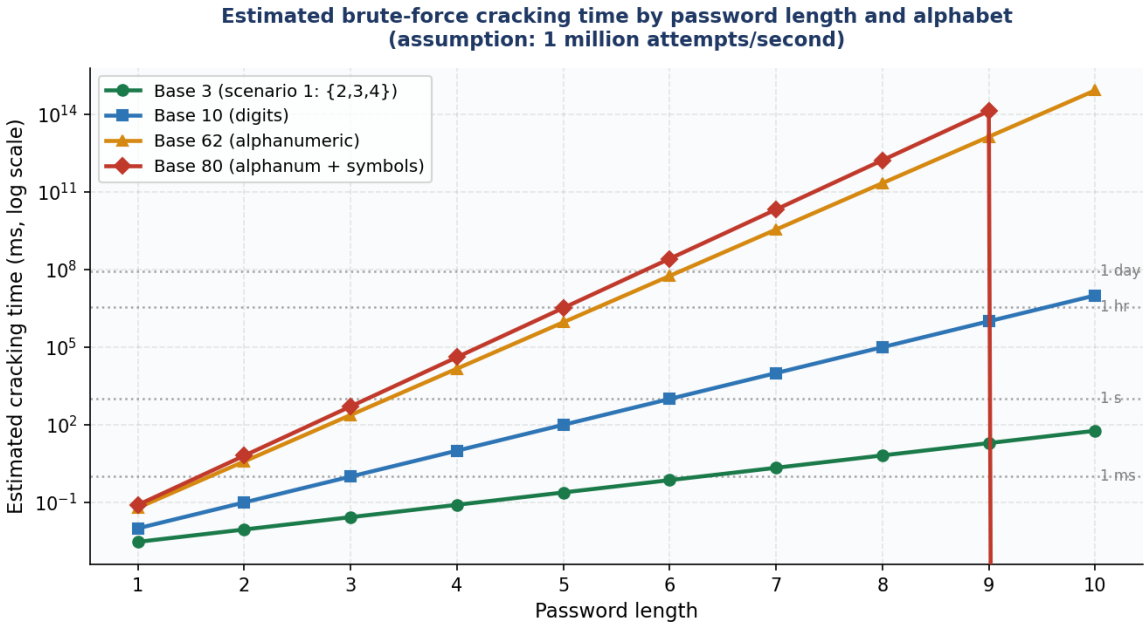


*Figure 4 — Estimated brute-force cracking time by password length, for different alphabet sizes (assumption: 1 million attempts/second)*

Figure 4 illustrates the exponential impact of length on cracking time. With an 80-character alphabet, an 8-character password would already require years of computation at 1M attempts/sec. These results confirm that length is the single most critical factor in resistance to brute-force attacks.

## 4.5 Man-in-the-Middle (MitM) Attack Simulation

A MitM attack occurs when a third party intercepts a communication between two entities without either party being aware. In EvilCorp, the simulation involves three entities:

- A (Alice): the legitimate sender;
- M (Mallory): the intercepting attacker;
- B (Bob): the legitimate recipient.

### Simulation Flow

| Step | Action | Entity |
|---|---|---|
| 1 | Alice sends a message intended for Bob | A → (network) → B |
| 2 | Mallory intercepts the message before it reaches Bob | A → M |
| 3 | Mallory reads and/or modifies the content | M |
| 4 | Mallory forwards a message (original or falsified) to Bob | M → B |
| 5 | Bob receives the message unaware of the interception | B |

In the C# implementation, the three entities are simulated by distinct functions exchanging strings through shared variables, illustrating the interception flow. The attacker can substitute the content before the final transmission.

## 5. Security Countermeasures

The attacks presented highlight the importance of sound security practices. The table below summarises appropriate countermeasures for each threat:

| Threat | Recommended countermeasures |
|---|---|
| Weak passwords | Mandatory complexity policy (min. 12 chars, uppercase, digits, symbols) |
| Dictionary attack | Salted hashing (bcrypt, Argon2) — renders pre-computed tables useless |
| Brute-force attack | Login attempt limiting, temporary lockout, CAPTCHA, progressive delays |
| Password storage | Never in plaintext; always hashed + salted with an adaptive-cost algorithm |
| MitM attack | TLS/SSL protocols, digital certificates, mutual authentication, HSTS |

## 6. Steganography

Steganography is the practice of concealing the very existence of a message by embedding it within an innocuous carrier (image, audio, video). Unlike encryption, it does not make the message unreadable — it makes it invisible.

© 2025 EvilCorp Project — For educational use only

## 6.1 LSB Method (Least Significant Bit)

The LSB method modifies the least significant bit of each byte in the carrier. For an image, each colour component (R, G, B) of a pixel is altered by at most 1 unit — a change imperceptible to the human eye — allowing message bits to be encoded.

### Encoding Principle

| Original pixel (R) | Message bit | Modified pixel (R) | Change |
|---|---|---|---|
| 10110100 (180) | 1 | 10110101 (181) | +1 — imperceptible |
| 11001010 (202) | 0 | 11001010 (202) | 0 — unchanged |
| 01110011 (115) | 1 | 01110011 (115) | 0 — LSB already correct |

To encode N message bits, a minimum of N pixels is required. A 100×100-pixel image (30,000 bytes) can conceal up to ~3,750 bytes of hidden data.

## 6.2 Implementation in EvilCorp

- The user loads a carrier image through the interface.
- The message is entered and converted into a bit stream.
- The LSBs of pixels are modified sequentially according to the message bits.
- An end-of-message marker is inserted to delimit the content during decoding.
- The modified image is saved; it is visually identical to the original.

## 6.3 Validation

| Carrier | Hidden message | Message size | Visual difference | Recovery OK? |
|---|---|---|---|---|
| PNG image 200×200 | "This is a test" | 15 chars | Imperceptible (ΔRGB ≤ 1) | ✓ |
| PNG image 100×100 | "EvilCorp2025" | 13 chars | Imperceptible (ΔRGB ≤ 1) | ✓ |

# 7. Tests and Validation

## 7.1 Encryption Algorithm Tests

| Algorithm | Plaintext | Key | Ciphertext | Decryption OK? |
|---|---|---|---|---|
| Caesar | HELLO | k = 3 | KHOOR | ✓ |
| Affine | AFFINE | a=5, b=8 | IHHWVC | ✓ |
| Hill (2×2) | HI | K=[[3,3],[2,5]] | TC | ✓ |

### 7.2 Attack Tests

| Attack | Target password | Result | Duration |
|---|---|---|---|
| Dictionary | 234 | ✓ Found (3 attempts) | < 1 ms |
| Brute force (scenario 1) | 324 | ✓ Found | < 1 ms |
| Brute force (scenario 2) | 83741 | ✓ Found | ~22 ms |
| Brute force (scenario 3) | aZ#1x! | ✗ Not completed | Timeout |

## 8. Conclusion

The EvilCorp project provided an opportunity to explore and experiment with several key concepts in computer security through a rigorous, hands-on approach. The main conclusions drawn from this work are:

- Classical algorithms (Caesar, Affine, Hill) are pedagogically valuable but cryptographically obsolete: they do not withstand modern attacks and should never be used to protect real data.
- Dictionary and brute-force attacks demonstrate the critical importance of a robust password policy. As shown by the complexity curves, length is the single most determining factor.
- The MitM simulation highlights the necessity of end-to-end encrypted protocols (TLS) for any sensitive communication.
- LSB steganography proves that information can be effectively concealed without any visible alteration of the carrier.

Future improvements could include the integration of modern algorithms (AES, RSA), the addition of a hashing simulation (SHA-256 + bcrypt), and a real-time frequency analysis interface to visualise the weaknesses of monoalphabetic substitution ciphers.

© 2025 EvilCorp Project — For educational use only